

Group 3

MBARARA UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF COMPUTING AND INFORMATICS



Program: Bachelor of Science in Computer Science

Module: Software Engineering

Assignment: Group Project

Lecturer: Keneth Baguma

Group number: BCS Group 3

Academic Year: 2025/2026

Year: 2

Group Members:

1. MUZOORA MORRIS	2024/BCS/229/PS
2. MUBIRU USAMA MALENDE	2024/BCS/103/PS
3. KAMUKAMA JORAM JOTHAN	2024/BCS/074/PS
4. ASIIMWE SHABELLAH	2024/BCS/049/PS
5. ASAASIRA LEILA	2024/BCS/046/PS
6. MUKUNDANE MEDARD	2024/BCS/110/PS

CAMPUS EVENT AND TICKETING SYSTEM (CETS) PROJECT REPORT

Table of Contents

1. Executive Summary	2
2. Project Background and Problem Statement	3
3. Objectives	3
4. System Architecture	4
5. Software Process Model	4
6. Functional Requirements	6
7. Non-Functional Requirements	6
8. System Features and Demonstration	7
8.1 Student / User Perspective	7
8.2 Event Organiser Dashboard	9
8.3 Administrator Panel	10
9. Testing	11
10. GitHub Repository	12
11. Future Work	13
12. Conclusion	14

1. Executive Summary

The Campus Event and Ticketing System (CETS) is a fully responsive, web based platform developed to address the systemic inefficiencies in campus event management. Prior to CETS, event advertisement relied on physical notice boards, informal messaging channels, and manual paper based ticket distribution which are processes that are slow, error prone, and inadequate for a growing student community.

CETS provides a centralized digital environment in which students can discover and book tickets for campus events, event organizers can create and monitor events in real time, and university administrators can oversee all platform activity, generate attendance reports, and ensure policy compliance.

The system is built using a Node.js backend, a MySQL relational database, and a modern responsive frontend. Development followed the Incremental Software Process Model, enabling

Group 3

iterative feature delivery and early testing. Security measures including password encryption and SQL injection prevention are embedded from the outset.

2. Project Background and Problem Statement

Mbarara University of Science and Technology hosts a rich calendar of academic, cultural, and sporting events throughout each academic year. The existing infrastructure for advertising these events and managing ticket sales presents several documented challenges:

- Event discovery is fragmented students rely on physical posters and whatApp groups which can be damaged or removed, and on informal messaging groups with limited reach.
- Manual ticket sales create long queues, administrative overhead, and a significant risk of ticket loss, duplication, or fraud.
- Organizers have no reliable mechanism for tracking ticket sales in real time or for communicating last minute changes to ticket holders.
- The university lacks a central repository of event attendance data, impeding strategic planning and institutional reporting.

These gaps formed the institutional background for CETS. The project was scoped to eliminate each of the above pain points through a purpose-built digital system.

3. Objectives

The primary objectives driving the design and development of CETS were:

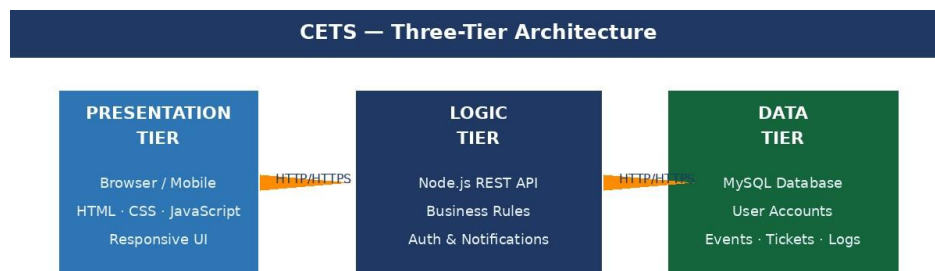
1. To reduce the time and manual effort required to create, advertise, and manage campus events.
2. To eliminate the risks associated with paper-based ticket sales, including duplication, loss, and fraud.
3. To provide students with a single, accessible platform for discovering and purchasing tickets for campus events.
4. To give event organizers real-time visibility into ticket sales, capacity, and attendee communications.
5. To enable university administrators to monitor platform activity, enforce content policies, and generate attendance reports.
6. To build a modular, scalable system capable of accommodating future enhancements such as QR code verification and integrated payment gateways.

Group 3

4. System Architecture

CETS is designed as a three tier web application. The three tiers communicate via HTTP/HTTPS requests. The backend acts as the sole gateway to the database, enforcing all security controls at this layer.

Frontend (Presentation)	Browser-based interface built with HTML, CSS, and JavaScript. Fully responsive across desktop, tablet, and mobile devices.
Backend (Logic)	Node.js RESTful API handling all business logic, authentication, event management, ticket booking, and notifications.
Database (Persistence)	MySQL relational database storing user accounts, event records, ticket transactions, and audit logs.



5. Software Process Model

The team adopted the Incremental Software Process Model. Rather than attempting to deliver all features in a single release cycle, development is structured into three increments. This approach allowed early usability feedback to be incorporated and ensured that each feature was rigorously tested before the next increment was initiated.

Increment	Focus	Deliverable
1	User Authentication	Secure login, registration, and role assignment (student, organizer, admin). Password hashing.

Group 3

2	Event Management	Create, edit, publish, and delete events. Discovery dashboard with search and category filtering.
3	Ticket Booking Engine	Real time ticket purchase, capacity enforcement, digital ticket generation, and organizer analytics.

Group 3

6. Functional Requirements

The following table summarizes the core functional requirements gathered during the analysis phase and maps each to its implementation status.

ID	Requirement	Priority	Status
FR-01	Users shall register and log in securely using unique credentials.	High	Implemented
FR-02	Organizers shall create events with title, date, location, capacity, and price.	High	Implemented
FR-03	Students shall browse, search, and filter events by category and date.	High	Implemented
FR-04	Students shall purchase tickets; the system shall enforce maximum capacity.	High	Implemented
FR-05	The system shall generate a unique digital ticket upon successful purchase.	High	Implemented
FR-06	The system shall mark events as 'Sold Out' when capacity is reached.	High	Implemented
FR-07	Organizers shall be able to edit event details postpublication.	Medium	Implemented
FR-08	The system shall notify ticket holders automatically when event details change.	Medium	Implemented
FR-09	Administrators shall moderate events and suspend accounts if necessary.	High	Implemented
FR-10	Administrators shall generate reports on ticket sales and attendance.	Medium	Implemented

7. Non-Functional Requirements

7.1 Performance

- Event listings and search results load within three seconds under concurrent user load.
- Ticket purchase transactions complete within five seconds, including database writes.

Group 3

7.2 Security

- All user passwords are hashed and salted before storage; plaintext passwords are never persisted.
- Input sanitization and parameterized queries prevent SQL injection attacks.
- Role based access control ensures students, organizers, and administrators see only what they are authorized to access.

7.3 Usability

- The interface follows established UX conventions consistent navigation, clear call-toaction buttons, and intuitive forms.
- The system is fully responsive and usable on screens from 360 px (mobile) to 1920 px (desktop).
- All critical user journeys (registration, event discovery, ticket purchase) require no more than four steps to complete.

7.4 Reliability and Availability

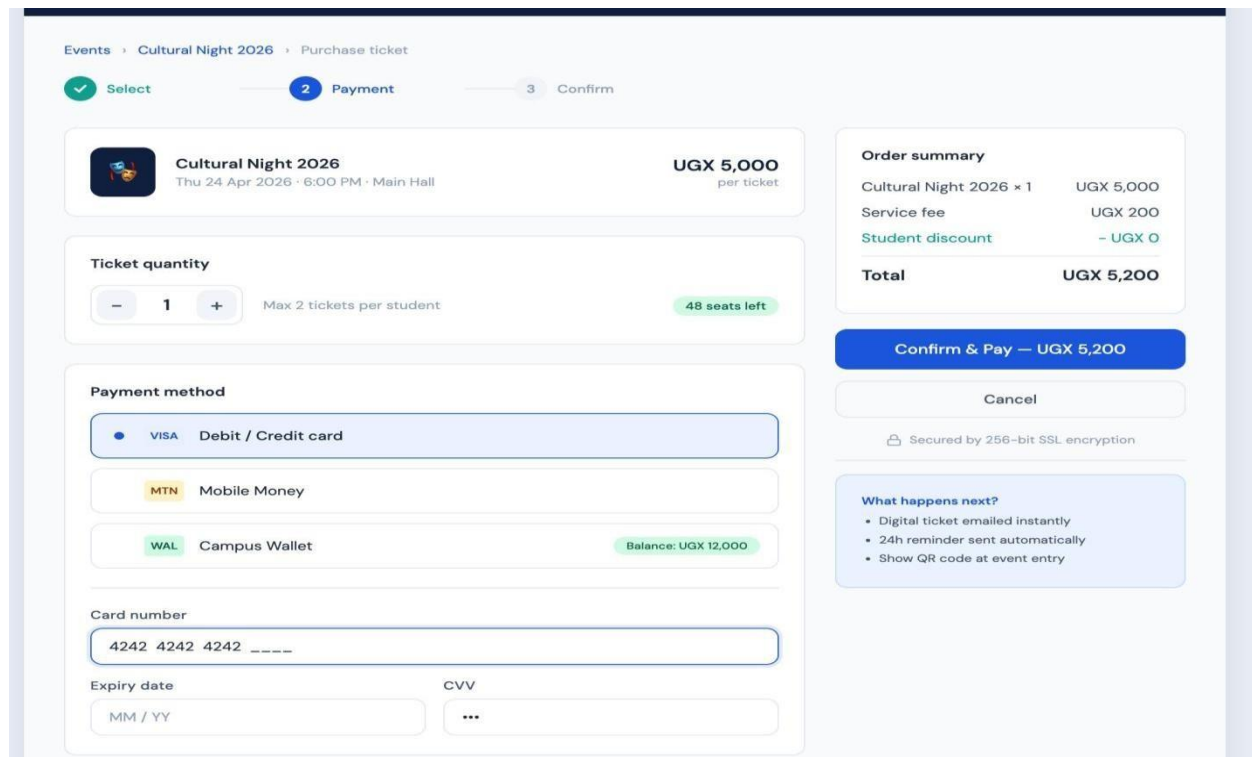
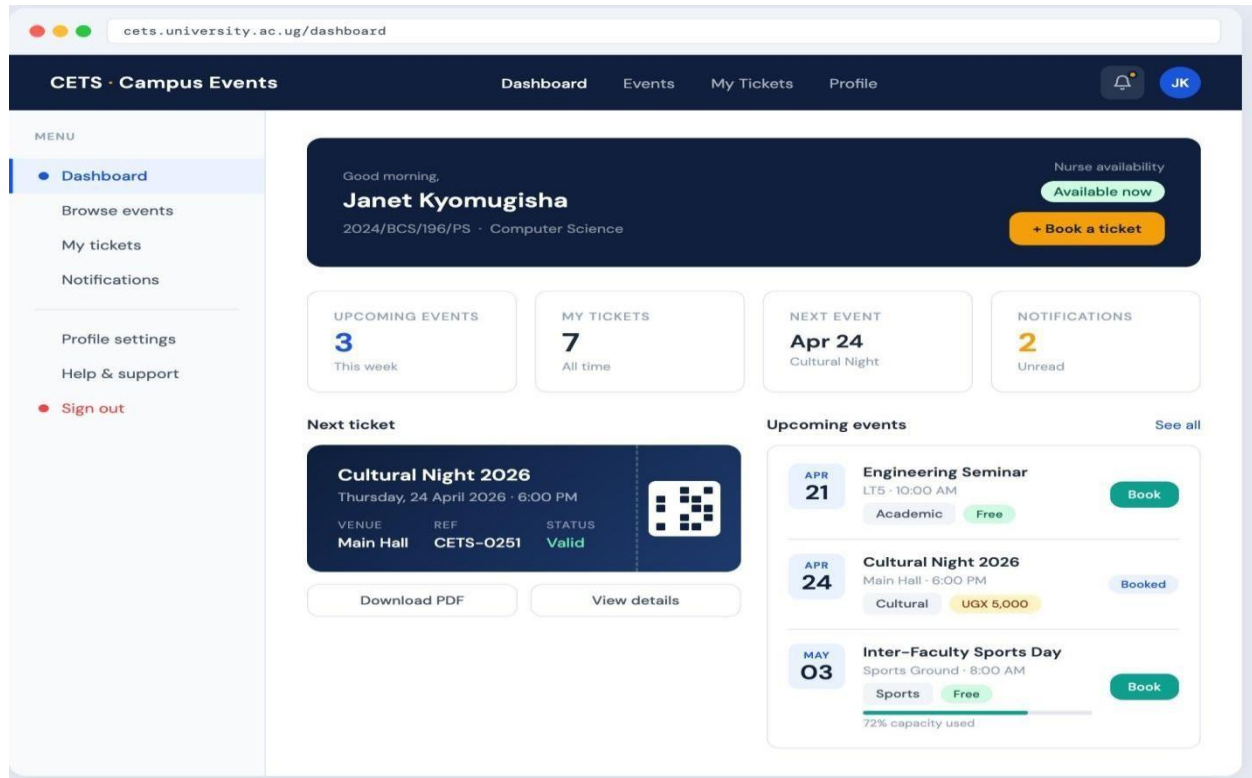
- The system logs all transactions to a dedicated audit table, ensuring a traceable record of all ticket sales and account actions.
- Database integrity is maintained through foreign key constraints and transactional writes.

8. System Features and Demonstration

8.1 Student / User Perspective

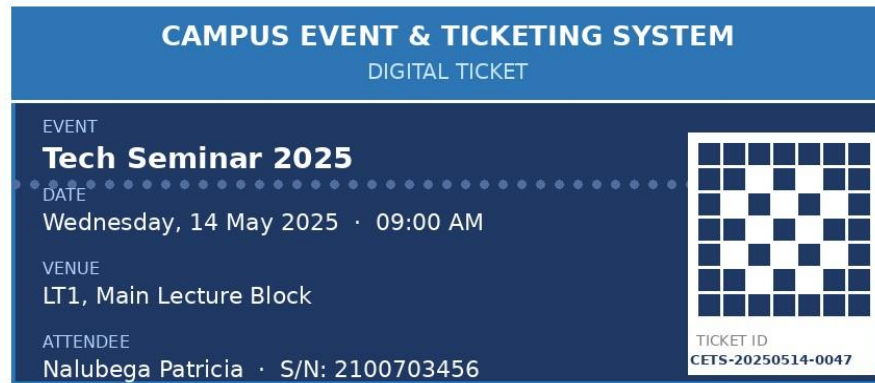
Students access CETS through a secure login page. Upon authentication, they are directed to the Discovery Dashboard, which displays all upcoming events in a card based layout. Events can be searched by keyword or filtered by category and date. Each card presents the title, date, venue, and remaining ticket count.

Group 3



Group 3

When the student clicks confirm and pay (and follows the steps afterwards), the backend atomically checks capacity, deducts one ticket, and generates a unique digital ticket record. The system automatically updates the status to 'Sold Out' and strictly prevents any further purchases once capacity is reached, fulfilling the maximum event capacity requirement.



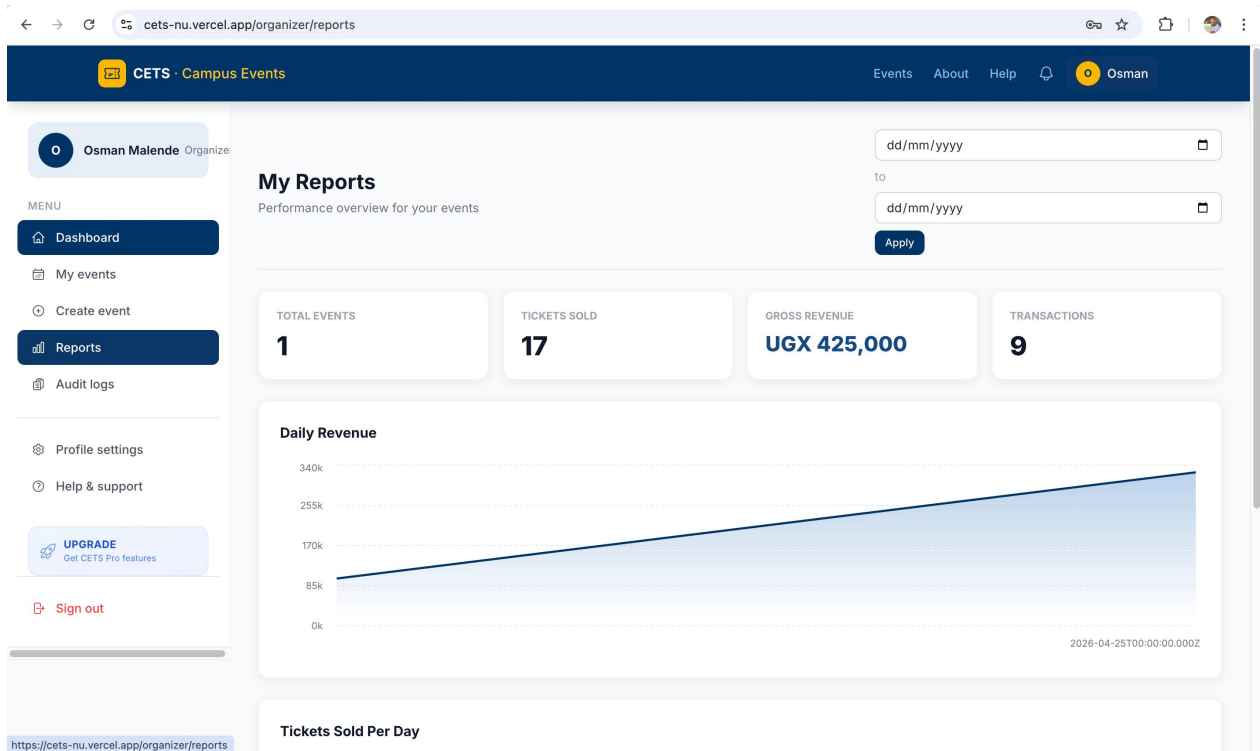
The way the ticket looks depends on the how the event organizer wants it

8.2 Event Organiser Dashboard

Organizers access a dedicated control panel from which they can create, edit, and monitor all events they have published. Creating an event is instant the organizer inputs the title, date, location, and sets the total capacity. Once published, it is immediately visible to all students on the discovery dashboard.

Should an organizer need to update the venue or timing, the system dispatches automated notifications to every user who has already purchased a ticket. Organizers also have access to a real-time analytics panel showing total tickets sold, tickets reserved, and remaining capacity.

Group 3

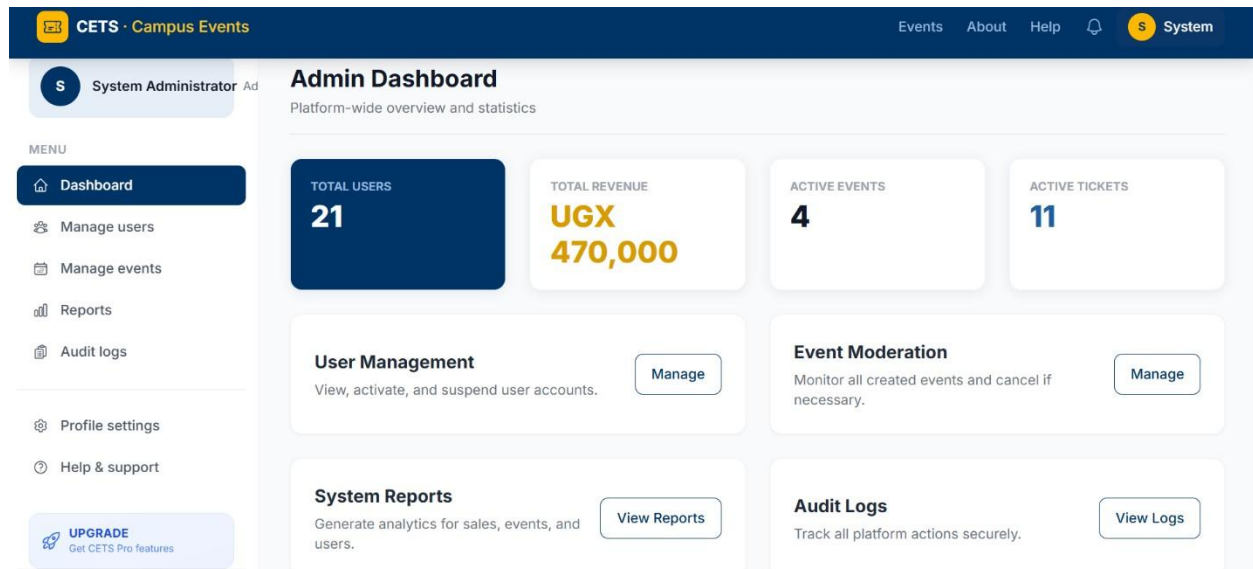


8.3 Administrator Panel

The Administrators have the highest level of access within CETS. The admin panel provides a global view of all events and user accounts on the platform. Administrators can approve, suspend, or remove events that violate university guidelines, and can manage organizer and student accounts.

The reporting module allows administrators to generate comprehensive reports covering ticket sales by event, attendance trends over a selected period, and historical engagement data. All actions are written to a secure audit log, providing a complete and tamper evident trail for troubleshooting or compliance purposes.

Group 3



9. Testing

CETS underwent both functional and non functional testing across all three increments.

9.1 Unit Testing

Individual backend functions including ticket reservation logic, capacity enforcement, and notification dispatch were tested in isolation to verify correct outputs for defined inputs.

9.2 Integration Testing

End to end test scenarios were executed to verify that the frontend, backend API, and database operated correctly as an integrated system. Key scenarios included the full ticket purchase flow and the event update-notification chain.

9.3 Peer Testing

The project was subjected to peer testing by a counterpart group. Reported issues relating to edge cases in the sold out enforcement logic and mobile layout inconsistencies were resolved before final submission.

Group 3

9.4 Performance Testing

Database queries were profiled and optimized to ensure that event listings load consistently within the three-second target, even under simulated concurrent user load.

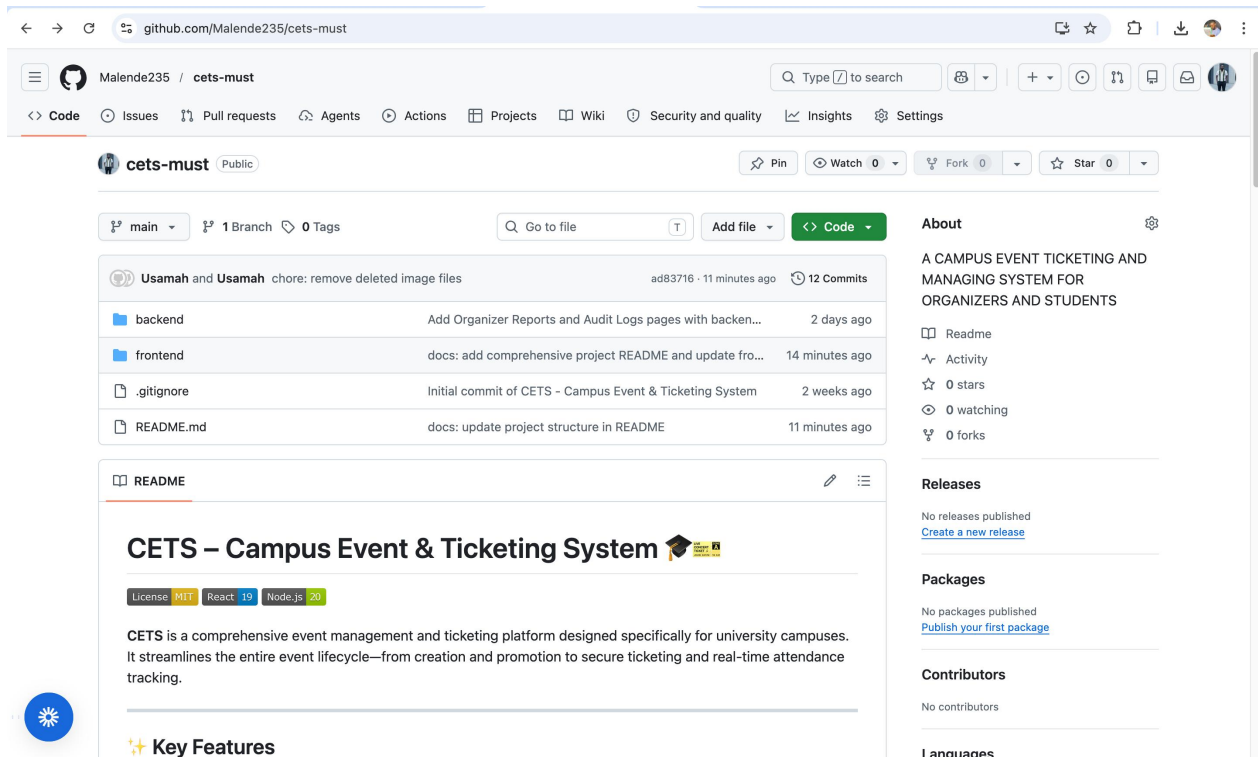
CETS – Testing Summary			
Test Type	Cases / Scope	Result	Outcome
Unit Testing	15 test cases	15 passed	0 failed
Integration Testing	8 scenarios	8 passed	0 failed
Peer / Cross-Group	12 observations	10 resolved	2 deferred
Performance Testing	3 load profiles	All < 3s	Target met
Security Testing	SQL injection, auth	All blocked	No vulnerabilities

Test Type	Tool / Method
Unit Testing	Manual test cases, isolated function calls
Integration Testing	End to end scenario walkthroughs
Peer Testing	Counterpart group review written test reports
Performance Testing	Database query profiling, load simulation
Security Testing	SQL injection probing, input validation checks

10. GitHub Repository

The complete project including source code, project report, demonstration video, and documentation is hosted on GitHub at the following URL: <https://github.com/Malende235/cets-must>

Group 3



Source Code	/src — Node.js backend, HTML/CSS/JS frontend, MySQL schema
Project Report	/report — This document in .docx and PDF formats
Demonstration Video	/video — 10-minute presentation video (MP4)
README	Setup instructions, dependency list, and run guide

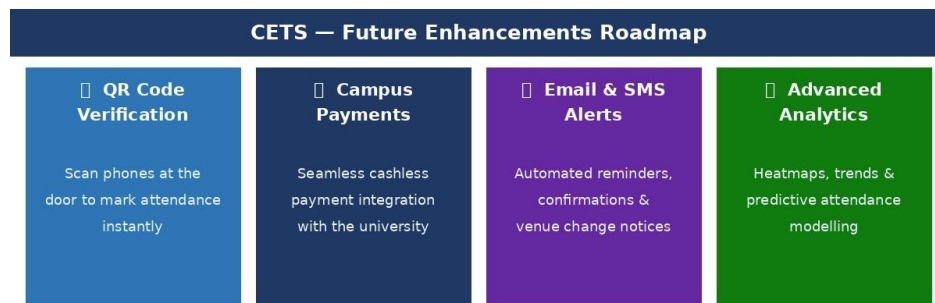
11. Future Work

CETS was architected with modularity in mind. The following enhancements have been identified for future development phases:

Feature	Description
---------	-------------

Group 3

QR Code Verification	Each digital ticket will embed a unique QR code. Organizers scan phones at venue entrances to verify attendance and update live attendance counts instantly.
Campus Payment Integration	Integration with the university's payment infrastructure will enable cashless, fully automated ticket payments without requiring external card portals.
Email & SMS Notifications	In addition to in app notifications, the system will dispatch email and SMS alerts for event reminders, ticket confirmations, and venue changes.
Mobile Application	A dedicated Android/iOS application will provide push notifications and an offline accessible digital ticket wallet.
Advanced Analytics	Richer reporting dashboards including heatmaps of peak booking times, demographic breakdowns, and predictive attendance modelling will support institutional planning.



12. Conclusion

The Campus Event and Ticketing System successfully addresses each of the business requirements identified at the outset of the project. CETS transforms a fragmented, manual process into a streamlined, transparent, and secure digital platform accessible to every member of the campus community.

By delivering a responsive web application with distinct role based experiences for students, event organizers, and administrators, CETS not only solves the immediate problems of poor event visibility and fraud prone ticket distribution, but also establishes a data-rich foundation for future engagement initiatives.

Group 3

The adoption of the Incremental Software Process Model, combined with rigorous peer-reviewed testing, ensured that each feature was stable, validated, and aligned with the stated requirements before deployment. The modular architecture guarantees that planned enhancements such as QR verification, payment integration, and mobile applications can be introduced with minimal disruption to the existing system.

CETS is not merely an academic exercise; it is a production ready digital service that modernizes university life, strengthens communication between students and organizers, and brings the campus community closer together through shared experiences.